PegaSold: Predicting Baku Home Prices with Decision Trees, Pegasos SVM and Bagging Built from Scratch

Abdulhalimova, Emiliya Asvarova, Aygun Bagirzade, Murad Karimli, Kenan

MLE-AI-201 Machine Learning, AI Academy — Cohort II

***Abstract*—**We predict asking prices of Baku residential listings (Task A, regression on log price) and a derived premium/standard price tier (Task B, classification) from the bina.az dataset. After cleaning 100,775 raw rows to 57,003 and building 82 features, we implement a decision tree, a Pegasos SVM and a bagging ensemble of trees in plain NumPy, and compare them with scikit-learn at matched settings. Our tree and SVM match the references (validation RMSE 0.2383 vs. 0.2388; accuracy 0.903 vs. 0.908). Bagging 30 trees is best on both tasks: test $R^2 = 0.886$ with a typical error of 14.9% for price, and test accuracy 0.934 with ROC-AUC 0.983 for the tier, a 20.9% lower test MSE than a single tree. Error is largest for listings above 1M AZN. Code: `github.com/b-murad/PegaSold` (tag `v1.0-final`).

***Index Terms*—**decision tree, support-vector machine, Pegasos, bagging, regression, classification, real-estate price prediction.

I. Introduction

Buyers and sellers in Baku have little way to judge whether an asking price is reasonable. We use the bina.az listing scrape to build two predictors: the price itself (Task A) and whether a listing is above or below the median price (Task B). The brief requires the decision tree and the SVM to be our own NumPy code, with scikit-learn used only as a baseline. Our contributions are: (i) a documented cleaning pipeline with 61 automated checks and no train/test leakage; (ii) from-scratch tree, Pegasos SVM and bagging ensemble that match scikit-learn; (iii) measured design choices (ensemble depth, SVM regularisation); and (iv) an error analysis showing where the models fail. A small web demo serves the trained models [1].

II. Related Work / Background

A decision tree greedily chooses the split that most reduces an impurity measure (Gini or entropy for classes, squared error for targets). Bagging averages trees fit to bootstrap samples, cancelling variance without changing bias. The soft-margin SVM minimises hinge loss plus an L_2 penalty; Pegasos [1] solves this primal problem with stochastic subgradient steps of size $1/(\lambda t)$, so no quadratic-programming solver is needed.

III. Data and Preprocessing

Source. The bina.az sale-listing scrape has 100,775 rows and 51 raw columns. We parse it into 19 working columns. Areas arrive as Azerbaijani text ("145 m^2"), so a parser extracts the number. The raw area column is in m^2 for flats but the land-plot area is in *sot* (1 sot = 100 m^2); we convert land area separately instead of mixing the two units.

Cleaning funnel. Table 1 lists each step. Most of the loss is deduplication: 36,990 rows are repeat scrapes of the same `item_id`, and we keep the latest scrape so no listing can appear in two splits. A further 544 "reposted twins" (same advert posted again under a new id) are removed. Scope keeps Baku and three residential categories (new build, old build, house/garden house). The domain filters drop rows with price outside 10,000–5,000,000 AZN, area outside 10–2,000 m^2, or price per m^2 outside 200–20,000 AZN. Filtered rows are removed, never clipped, and the luxury tail is kept.

Table 1. Cleaning funnel (counts recorded in `preprocessing_report.json`).

Step	Rows	Removed
Raw scrape	100,775	—
Deduplicate	63,241	37,534
Scope (Baku, residential)	57,069	6,172
Value windows (price, area)	57,003	66

Targets and split. Task A predicts $\log(\text{price})$. Task B is a premium/standard tier, premium meaning price $\geq$ the *training-set* median, 209,000 AZN. The cleaned rows are split 70/15/15 with seed 42 into 39,902 train, 8,550 validation and 8,551 test rows. Premium share is 0.500, 0.505 and 0.501 in the three splits. Split identity hashes are stored so a re-run is provably the same split.

Features and leakage. The model matrix has 82 columns: 11 continuous (log area, log land, rooms, latitude, longitude, floor, building floors, distance to the city centre, floor ratio, area per room, log land ratio), 9 binary flags (floor known, repaired, title deed, mortgage, VIP, featured, owner-sold, ground floor, top floor), 3 property-type indicators and 59 location indicators (58 locations with at least 100 training rows, plus "other"). Every statistic used to transform data (median fills, scaling mean and standard deviation, the tier threshold, the location list) is measured on the training split only. We removed three columns that contain the answer: `unit_price`, `total_price` and `description`. An audit confirms none survives into the features and that no numeric feature has $|r| > 0.95$ with price (largest: rooms, 0.53).

IV. Method

A. Decision tree (from scratch, NumPy)

The tree is grown recursively. At every node, `_best_split` tries every threshold, midway between neighbouring sorted values, on every feature and keeps the split with the largest size-weighted impurity reduction. Classification uses Gini

or entropy and regression uses mean squared error; a leaf predicts the majority class or the mean target. Growth stops at the depth cap, when a node has fewer rows than the minimum, or when it is pure. We also enforce a minimum leaf size and a minimum impurity decrease. Classification leaves store their class counts, so `predict_proba` returns class frequencies; this is what makes ROC-AUC meaningful. Each split node records its size and gain, giving impurity-based feature importances.

B. Support-vector machine (Pegasos)

We minimise the primal soft-margin objective with the Pegasos sub-gradient method,

$$J(\mathbf{w},b) = \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_{i=1}^{n} \max\bigl(0,\, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\bigr), \quad (1)$$

visiting one listing at a time with step size $\eta_t = 1/(\lambda t)$. A misclassified or in-margin point pushes the boundary; regularisation shrinks $\mathbf{w}$ but not the intercept b, whose step is capped at 1. After every update $\mathbf{w}$ is projected back onto the ball $\|\mathbf{w}\| \le 1/\sqrt{\lambda}$, as in the original algorithm [1]. Features are already standardised from training statistics. We track the objective per epoch and report support vectors and the margin width $2/\|\mathbf{w}\|$. Because Pegasos optimises hinge loss rather than accuracy, we also tune the decision threshold on validation.

C. Nonlinear SVM (random Fourier features)

To test a nonlinear boundary without a kernel solver, we map the scaled features through 128 random Fourier features, $\sqrt{2/D}\cos(\mathbf{x}^\top W + \mathbf{b})$ with $W \sim \mathcal{N}(0, 2\gamma)$, which approximates an RBF kernel, and train the same Pegasos solver on the mapped data. γ is chosen from $\{0.01, 0.1, 1\}$ on validation using an 8,000-row training sample; an RBF `SVC` on the same rows is the reference.

D. Bagging ensemble

`BaggedTrees` trains 30 of our trees, each on 6,000 rows drawn with replacement. Regression averages the members. Classification uses soft voting: it averages class probabilities rather than counting votes, and handles a bootstrap sample that misses a class. Out-of-bag scoring judges each training row only by the trees that never saw it, giving a held-out estimate without touching validation data. We grow members to depth 16, deeper than the single tree's tuned depth, because averaging controls variance. Feature subsampling, as in a random forest, was tried and did not help here, so this is plain bagging.

V. Experimental Setup

Metrics. Task A: MSE, RMSE, MAE and R^2 on log price, plus a typical error in percent, $e^{\mathrm{MAE}} - 1$. Task B: accuracy and ROC-AUC.

Tuning. Tree depth is swept over $\{2, 4, \ldots, 16, 20\}$ and the SVM λ over $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$, both on a fixed 6,000-row training subsample and chosen on validation, because our exact tree is slow. Final models are then trained on all 39,902 rows. The test set is scored once, at the end.

Baselines. scikit-learn is used only for comparison, with matched hyper-parameters: a decision tree at the same depth and leaf size, Ridge regression, a linear SVM (`LinearSVC`), and a dummy predictor. We also run 5-fold cross-validation on the full training split for the tree (both tasks) and the SVM, each against its scikit-learn counterpart. All seeds are fixed (42); repeated runs give identical numbers.

Table 2. Task A, regression on log price (RMSE lower is better).

Model	RMSE val	RMSE test	R^2 test	Err. test
Dummy (mean)	0.632	—	—	—
Ridge	0.241	—	—	—
scikit-learn tree	0.239	—	—	—
Ours: tree (depth 10)	0.238	0.245	0.856	17.8%
Ours: bagging	**0.205**	**0.218**	**0.886**	**14.9%**

Table 3. Task B, premium vs. standard tier.

Model	Acc val	AUC val	Acc test	AUC test
Dummy	0.494	—	—	—
LinearSVC	0.908	0.970	—	—
scikit-learn tree	0.911	0.967	—	—
Ours: tree (depth 8)	0.911	0.966	0.916	0.970
Ours: SVM	0.903	0.967	0.908	0.970
Ours: bagging	**0.929**	**0.981**	**0.934**	**0.983**

VI. Results

Table 2 and Table 3 give the headline numbers. Baselines were run on validation only.

Our code matches the reference. Our tree is within 0.0005 RMSE of scikit-learn on validation (0.2383 vs. 0.2388) and within 0.0002 accuracy on Task B (0.9113 vs. 0.9111). Our Pegasos SVM reaches 0.903 accuracy against 0.908 for the exact linear solver, and the same AUC to within 0.003.

Bagging helps both tasks. Averaging 30 trees cuts regression MSE from 0.0568 to 0.0421 on validation (25.9%) and from 0.0598 to 0.0473 on test (20.9%). For classification it lifts validation AUC from 0.9664 to 0.9808. Out-of-bag estimates, which use no validation data, are $R^2 = 0.888$ and accuracy 0.929, close to the validation scores, which is independent evidence that the ensemble is not overfitting.

Two measured design choices. (i) Using the single tree's swept depth (8) for ensemble members was a mistake: at depth 8 the ensemble scores accuracy 0.914, AUC 0.976, OOB 0.916; at depth 16 it scores 0.929, 0.981, 0.929, a gain of 1.4 points of accuracy. (ii) Lowering the SVM λ from 0.01 to 0.001 raised validation accuracy from 0.890 to 0.903 and AUC from 0.957 to 0.967.

Cross-validation and nonlinear SVM. Five-fold cross-validation agrees with the held-out numbers and with scikit-learn: tree log-RMSE 0.246 ± 0.003 (scikit-learn 0.246 ± 0.004), tree accuracy 0.909 ± 0.001 (both), Pegasos accuracy 0.904 ± 0.002 (LinearSVC 0.904 ± 0.002). The random-Fourier-feature SVM ($\gamma = 0.01$, 8,000 rows) reaches test accuracy 0.842 and AUC 0.921, below both the RBF SVC on the same rows (0.875, 0.955) and the linear SVM (0.908, 0.970): a nonlinear boundary did not help here at this sample size. The regression tree's importances are dominated by log area (0.63) and distance to the city centre (0.24).

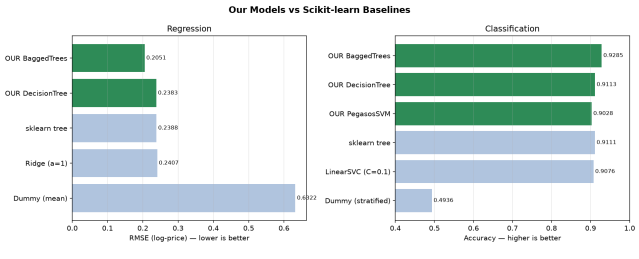


Figure 1. Our models against the scikit-learn baselines.

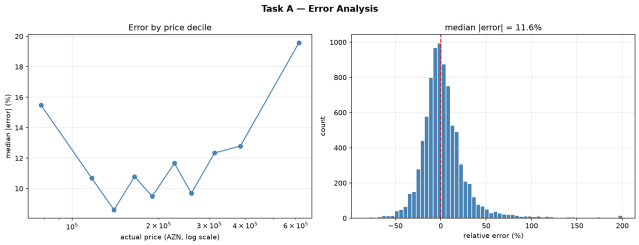


Figure 2. Task A error by price decile (validation).

VII. Discussion and Limitations

Where the model fails. Error is not uniform (Fig. 2). For the single tree on validation, the median error is 10.2% for listings between 100k and 350k AZN, about 15.6% for 350k–1M, and 33% above 1M (117 listings); the cheapest listings (below 100k) also reach a 15.5% median. Luxury properties are rare, strongly priced by unobserved features such as view and finish, and the tree can only average a few similar training rows.

Bias and variance. A single deep tree overfits, which is why averaging trees helps: it removes variance and leaves bias where it was. The SVM is linear, so it cannot use feature interactions, yet matches the tree's ranking quality (AUC 0.970 for both).

Accuracy vs. ranking. A shallow tree has impure leaves and gives graded probabilities; a deep tree has nearly pure leaves and outputs values near 0 or 1. Accuracy ignores this, but AUC measures ranking, so the depth that maximises one does not maximise the other. Bagging removes the trade-off.

Limitations. (1) Depth and λ are tuned on 6,000 rows but used on 39,902, so the chosen depths are probably slightly shallow. (2) Ensemble depth is fixed by one measurement, not its own sweep. (3) Our tree rechecks every threshold from scratch, so a full fit takes seconds to minutes; sorting each feature once and sweeping with running totals would be much faster. (4) The data are asking prices from one scrape, not sale prices. (5) Our `min_impurity_decrease` compares the raw gain, whereas scikit-learn scales it by node size, so the same value prunes differently.

VIII. Conclusion

We built a decision tree, a Pegasos SVM and a bagging ensemble from scratch on 57,003 cleaned Baku listings. The tree and SVM match scikit-learn, and bagging gives the best results on both tasks: $R^2 = 0.886$ and 14.9% typical error for price, 93.4% accuracy and AUC 0.983 for the tier. Next steps would be a proper sweep of ensemble depth and size, a faster split search, and per-segment models for luxury listings.

Tools and Acknowledgements

The tree, SVM and ensemble are our own NumPy code. scikit-learn is used only for baselines and correctness checks, as the brief allows. An AI coding assistant (Claude Code) was used to help write unit tests, documentation, the web demo and drafts of this report; the team reviewed all code and checked every number against the files in `reports/`. Data: bina.az listings supplied with the course.

References

[1] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.